

Redis Architecture

System Design Deep Dive - Complete Detailed Notes

Every concept explained with an example and a diagram

#	Chapter	Core question it answers
1	What is Redis and Why Does it Matter	Why use an in-memory store?
2	Capacity Estimation	How much can one Redis instance handle?
3	RESP: the Wire Protocol	How do client and server talk?
4	The Event Loop and epoll	How can ONE thread serve 100,000 clients?
5	The In-Memory Storage Engine	How does Redis store data so compactly?
6	Persistence: RDB, AOF, Hybrid	How much data survives a crash?
7	End-to-End Walkthrough	What happens on SET user1 Alice?
8	Lazy Free	How is a huge delete done without blocking?
9	Big Picture, Cheat Sheet and Interview Q&A	How does it all connect?

1. What is Redis and Why Does it Matter?

Redis is an important component in almost all real-world architectures, and its internal architecture is a very common **interview topic**. This chapter explains what it is and why we need it.

1.1 Definition: an in-memory key-value store

- **Key-value store:** data is stored as pairs, a key (name) pointing to a value. Example: key user :1 -> value Alice.
- **In-memory:** Redis keeps **all data in RAM** rather than on persistent storage (disk).
- It is designed for **extreme speed and very low latency**.

EXAMPLE

Think of a restaurant. The database is the big warehouse in the basement (huge, but slow to walk to). Redis is the small shelf right next to the chef holding today's most-used ingredients: tiny, but you grab things instantly.

Why RAM matters: typical access times (illustrative, not to scale)

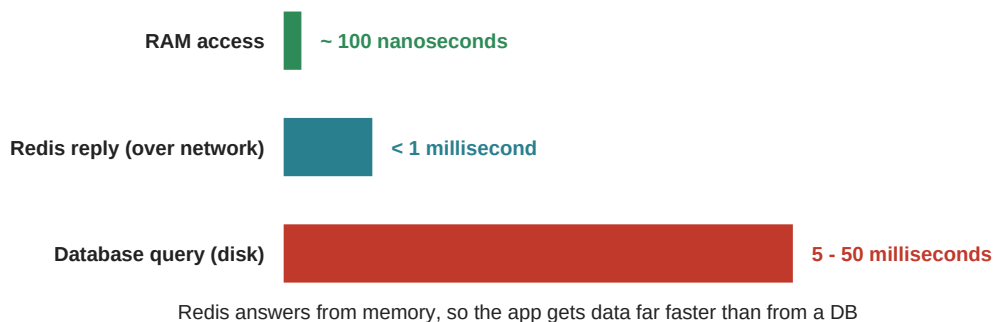


Figure 1: RAM-based Redis answers orders of magnitude faster than a disk-based database

1.2 Why do we need Redis? (classic interview question)

Because we want **low latency and fast speed**, and Redis delivers that **because it stores everything in RAM**.

EXAMPLE

A product page is requested 5,000 times per second. Reading it from the database each time takes 30 ms and overloads the DB. Reading it from Redis takes well under 1 ms, so the page is faster and the DB is protected.

1.3 Where Redis sits: a cache between app server and database

Redis acts like a cache that sits **between your database and your app server**.

- The client sends a request to the app server.
- The app server **first checks Redis**.
- **Cache hit:** data is in Redis, so it is returned instantly.
- **Cache miss:** data is not in Redis, so the app fetches it from the DB, **stores it in Redis**, then returns it.

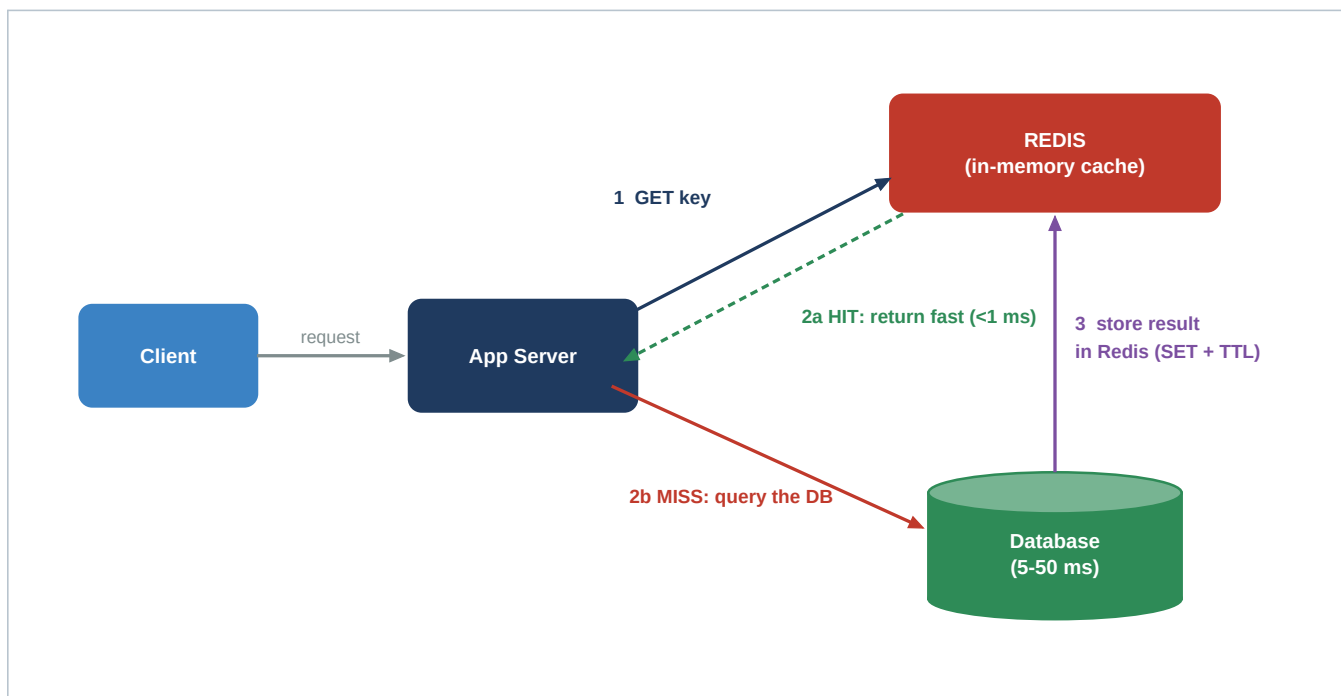


Figure 2: Request flow with Redis as a cache layer: hit vs miss

1.4 Redis is a cache, NOT the primary database

Redis data is **volatile**: it lives in RAM, and when the machine loses power or the process dies, the data is gone. Redis does have persistence and replication features (covered later), but for design purposes treat it as a **caching layer**, not as your source of truth.

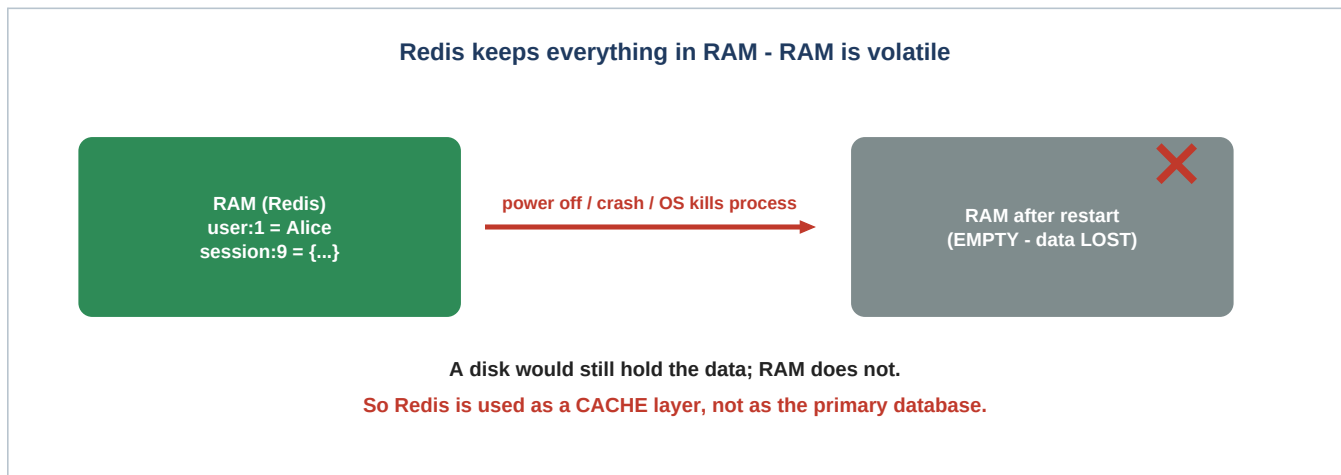


Figure 3: RAM loses its contents on power-off, unlike disk

EXAMPLE

Your e-commerce site keeps orders in MySQL (the primary DB) and caches popular product pages in Redis. If Redis restarts, the cache is just empty and warms up again from MySQL. Nothing important is lost. If you stored the orders only in Redis, a crash could erase them.

KEY TAKEAWAY

Redis = fast, in-memory, volatile. Use it for caching, sessions, leaderboards, counters; keep the permanent truth in a real database.

2. Capacity Estimation

Capacity estimation is an important part of any system design: without it you cannot decide how many servers or instances you need. Three numbers matter for Redis: throughput, latency and memory.

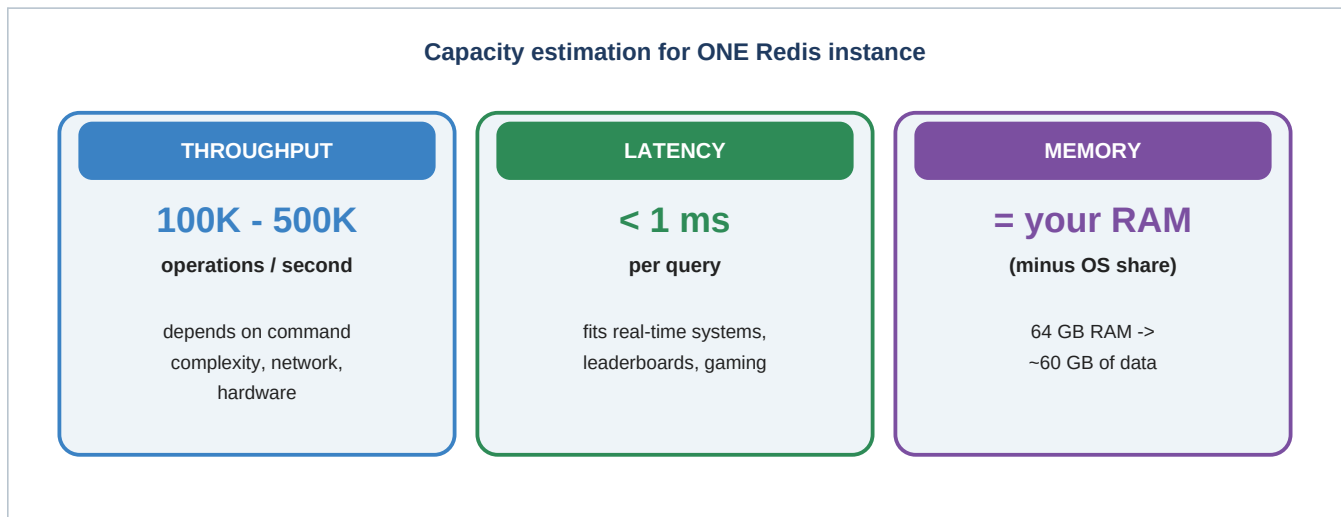


Figure 4: The three capacity numbers for one Redis instance

2.1 Throughput

One Redis instance can handle roughly **100K to 500K operations per second**. What decides the number:

- **Command complexity:** a simple command like GET is processed very fast; a heavy command that touches lots of data takes longer.
- **Network:** usually the main limiter, though network speed is rarely a barrier these days.
- **Hardware:** CPU speed and memory.

EXAMPLE

Worked example: your app needs 1,000,000 cache reads per second at peak. One instance handles at most about 500K, so you need **at least 2 instances** (more for safety headroom, e.g. 4). Simple GETs would hit the high end of the range; heavy commands would hit the low end.

2.2 Latency

Redis answers a query in **less than 1 millisecond**. That is why it is used in **real-time systems, leaderboards and gaming**.

EXAMPLE

In an online game, after every match the scoreboard must refresh immediately for millions of players. A sorted set in Redis updates and returns the top 10 in under a millisecond.

2.3 Memory usage

Everything is stored in RAM, so **how much data Redis can hold depends on how much RAM you have**. The operating system also needs some RAM.

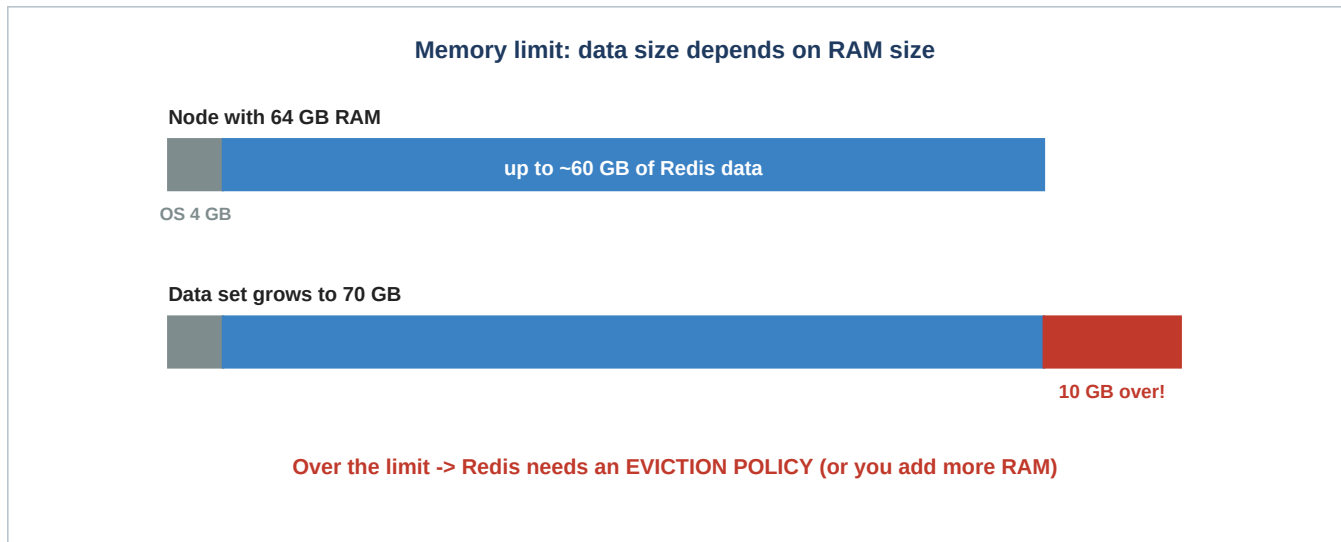


Figure 5: A 64 GB node holds about 60 GB of data; a 70 GB dataset overflows

EXAMPLE

Node with 64 GB RAM, OS needs about 4 GB -> about **60 GB** usable for Redis data. If your dataset grows to **70 GB**, 10 GB cannot fit, so Redis needs an **eviction policy** (or you add RAM / more nodes).

2.4 Eviction policies

- **LRU (Least Recently Used)**: remove the keys that were used the longest time ago. Good for **sessions**: inactive sessions can go.
- **LFU (Least Frequently Used)**: remove the keys used the fewest times overall.
- **No eviction (over-provision)**: for **leaderboards or counters** you usually do not want any key deleted; instead buy more RAM so everything fits.

Choosing what to do when memory is full



LFU = Least Frequently Used (evict items accessed the fewest times) is the other common policy

Figure 6: Pick the eviction approach according to the kind of data

EXAMPLE

Sessions + LRU: a user who has been idle for two days loses the session and simply logs in again: harmless.

Leaderboard + no eviction: deleting a player's score because memory was full would be a visible bug.

In Redis configuration these choices are set with the `maxmemory` limit and `maxmemory-policy` (for example `allkeys-lru`, `allkeys-lfu`, `noeviction`).

3. RESP: the Redis Serialization Protocol

Before looking at the end-to-end architecture, we need the core components that make Redis fast. The first is **RESP**, the **wire format** Redis uses to communicate between client and server, normally over a **TCP connection**.

3.1 What RESP does

When your application sends a command such as `SET user1 Alice`, the client library converts it into a **simple, human-readable text format** that is easy to parse in any programming language. Every command the app sends and every reply Redis returns travels as RESP bytes over a TCP socket.

3.2 Type prefix characters

RESP: the first character tells the data type

Prefix	Type	Example on the wire	Meaning
+	Simple string	+OK\r\n	success reply
-	Error	-ERR unknown command\r\n	something went wrong
:	Integer	:1000\r\n	a number
\$	Bulk string	\$5\r\nhello\r\n	length, then the bytes
*	Array	*3\r\n ...	count of elements that follow
\r\n	CRLF	(carriage return + line feed)	end-of-line marker between parts

Plain text, human-readable, trivial to parse in any language, travels over a TCP socket

Figure 7: The first character of each RESP element tells its type

EXAMPLE

+OK\r\n = simple string 'OK' (success). -ERR unknown command\r\n = error. :1000\r\n = the integer 1000 (for example the result of a counter increment). \$5\r\nhello\r\n = a 5-byte string 'hello'. *3\r\n... = an array of 3 elements follows.

\r\n (carriage return + line feed) is the **end-of-line marker** placed between the parts of a message.

3.3 Worked example: encoding SET user1 Alice

- The command has **three elements**: SET, user1, Alice. So it is an **array** of 3: *3\r\n.
- **SET** is a bulk string of length 3: \$3\r\nSET\r\n.
- **user1** is a bulk string of length 5 (u, s, e, r, 1): \$5\r\nuser1\r\n.
- **Alice** is a bulk string of length 5: \$5\r\nAlice\r\n.
- Concatenated into **one line** on the wire.

Encoding the command SET user1 Alice in RESP

array header	<code>*3\r\n</code>	array of 3 elements (SET, key, value)
element 1: SET	<code>\$3\r\nSET\r\n</code>	bulk string, length 3
element 2: user1	<code>\$5\r\nuser1\r\n</code>	bulk string, length 5 (u,s,e,r,1)
element 3: Alice	<code>\$5\r\nAlice\r\n</code>	bulk string, length 5 (A,l,i,c,e)

On the wire (ONE continuous line):

```
*3\r\n$3\r\nSET\r\n$5\r\nuser1\r\n$5\r\nAlice\r\n
```

Redis replies with a simple string: `+OK\r\n`

Figure 8: Step-by-step RESP encoding of SET user1 Alice and the +OK reply

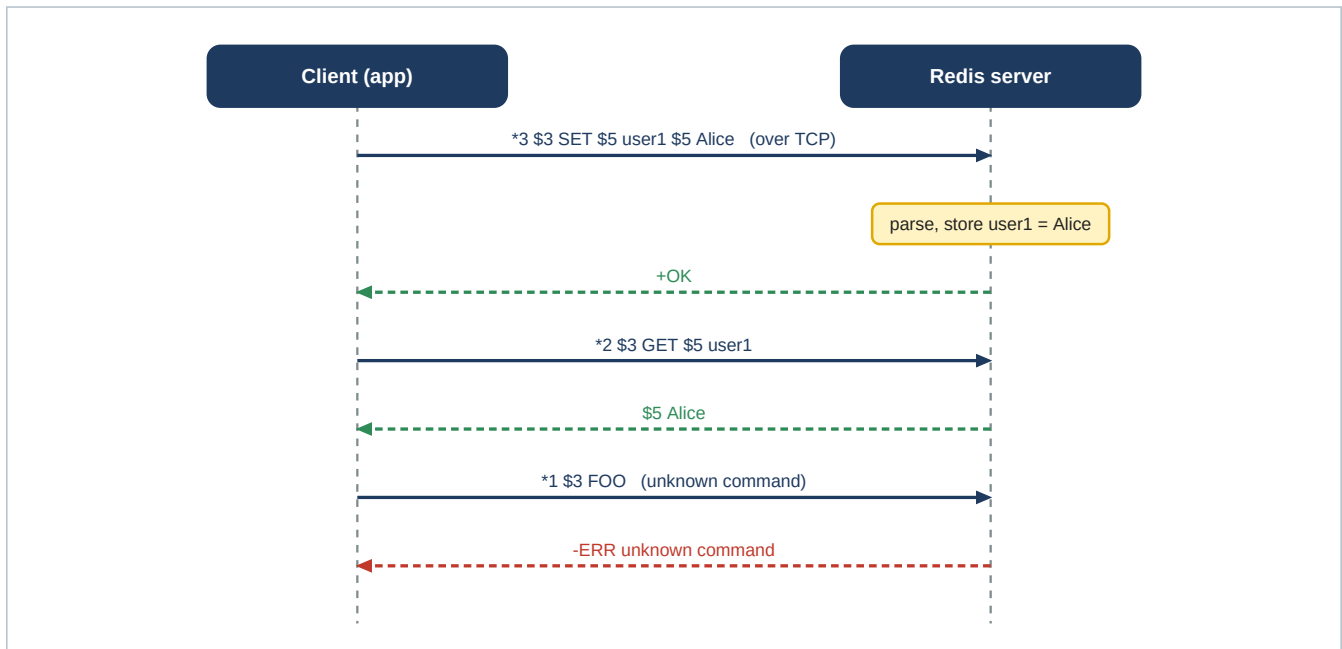


Figure 9: A SET, a GET and an error, as seen on the wire

KEY TAKEAWAY

RESP = a tiny text protocol with a type prefix and CRLF terminators. Simple to parse means fast to parse, which is one of the reasons Redis is quick.

4. The Event Loop and epoll

4.1 Redis executes commands on a single thread

Most real-world servers are heavily multi-threaded, yet Redis is hugely popular and **single-threaded for command execution**. How can one thread serve tens of thousands of connections without getting blocked? First, why not use threads?

4.2 Why not one thread per client?

Naive thinking: 100,000 clients connected, so create one thread per client. That is disastrous for Redis because each thread needs a **stack**, **CPU scheduling**, **context switching** and **synchronization** (locks).

EXAMPLE

Assume each thread uses a 1 MB stack. 100,000 clients x 1 MB = **100 GB of RAM** spent only on thread stacks, while RAM is exactly the scarce resource Redis lives on (and your data also needs space). That is not feasible.

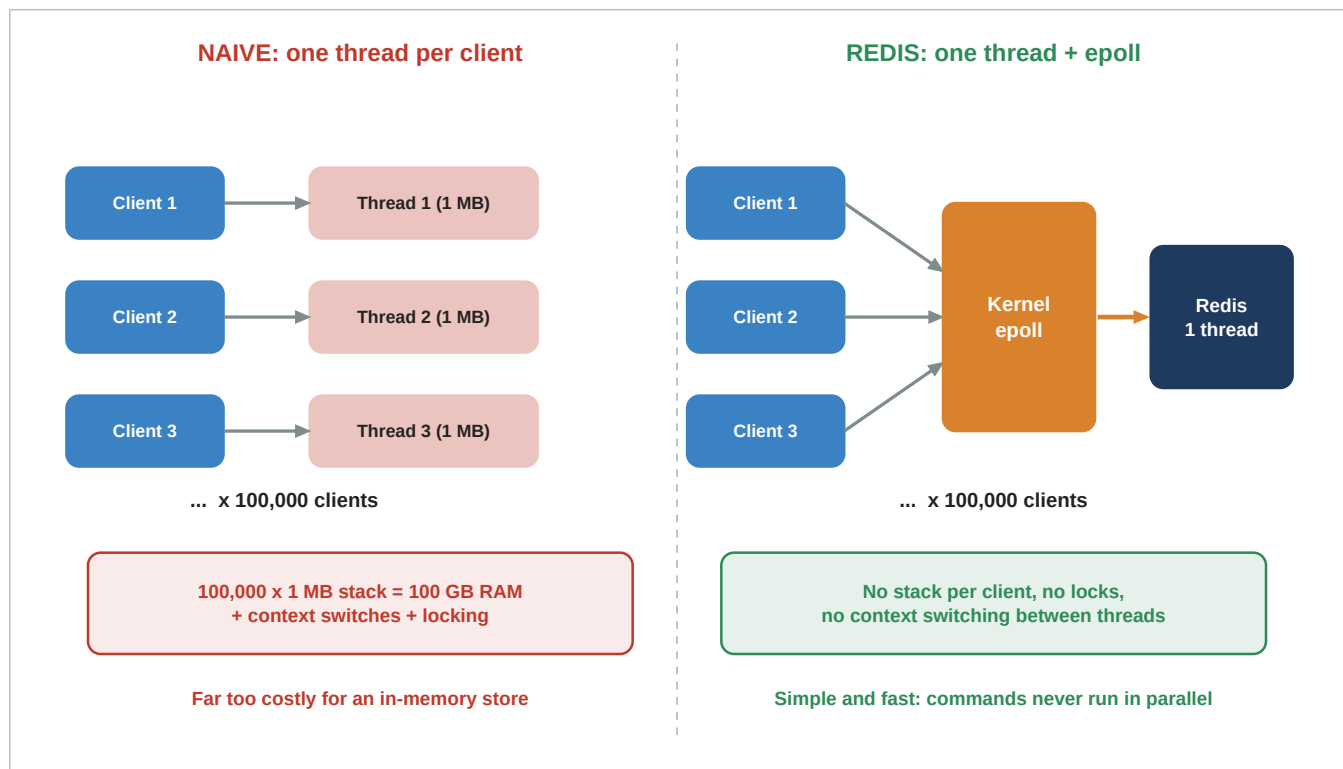


Figure 10: Thread-per-client vs Redis's single thread with epoll

Using **one thread** removes all of this overhead: no per-client stacks, no locks, no context switches between threads. Commands run one after another, so they cannot interfere with each other.

4.3 epoll: how one thread avoids being blocked

Redis opens many sockets. Instead of checking thousands of sockets itself in a loop (a huge waste of CPU), it calls **epoll** (`epoll_wait`), a **Linux kernel feature**:

- Redis registers all its sockets with the kernel and **goes to sleep**.
- The **kernel keeps track of the sockets**.
- When data arrives on any socket, the kernel **wakes Redis up** and returns the **list of ready sockets**.

- Redis processes **only those** sockets.

WITHOUT epoll: Redis would keep checking every socket itself (wasteful busy loop)



WITH epoll: the KERNEL watches the sockets and reports only the ready ones

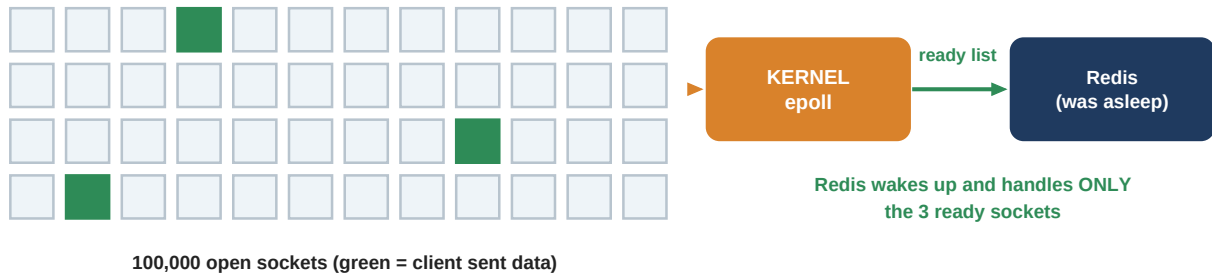


Figure 11: Busy-checking every socket vs letting the kernel report only the ready ones

EXAMPLE

100,000 sockets are open. Five clients send requests. The kernel wakes Redis and says 'sockets 17, 42, 311, 9,020 and 55,001 are ready'. Redis handles just those five, then goes back to sleep. Client A and B (idle) cost nothing; Client C asked GET user : 123, so its socket is reported ready and served.

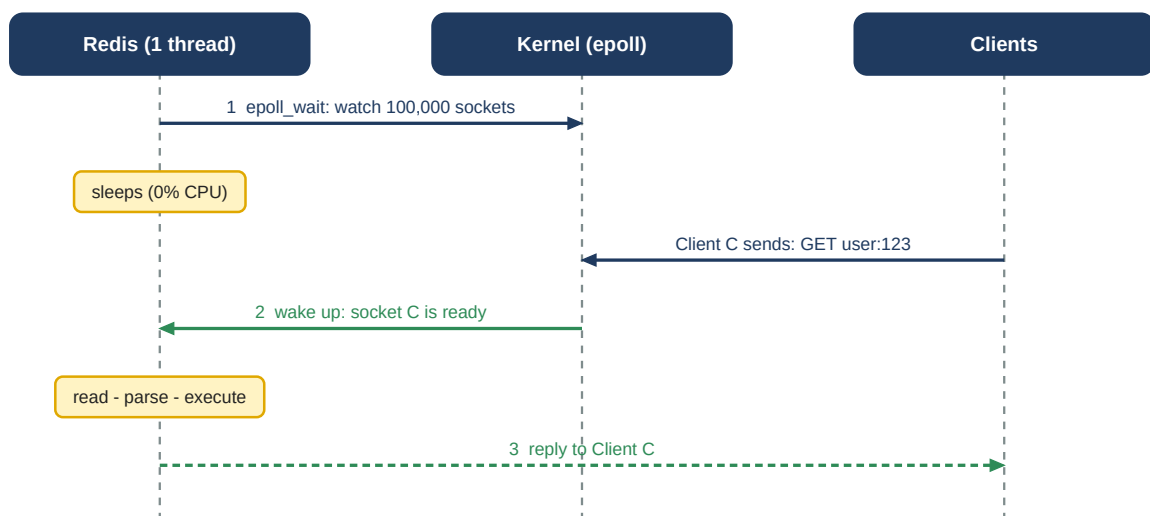


Figure 12: Sequence: Redis sleeps in `epoll_wait` until the kernel reports a ready socket

4.4 The event loop (reactor pattern)

A common confusion: **epoll is not the event loop**. The **event loop** is the higher-level **reactor pattern** implementation (the same pattern Node.js uses) that **uses epoll** to find ready sockets and then does the real work:

- while (true) loop forever:
- 1) **epoll_wait** for ready sockets.
- 2) **Read** the request.
- 3) **Parse** the command (RESP).
- 4) **Execute** the command.
- 5) **Send** the response.

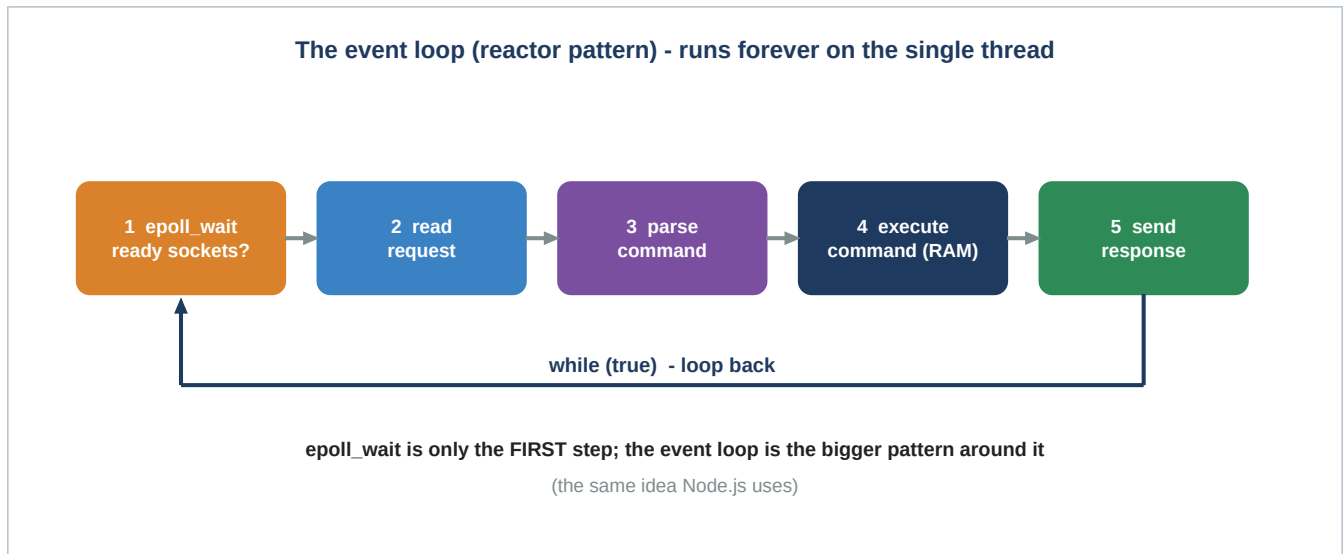


Figure 13: The event loop: epoll is just step 1

KEY TAKEAWAY

Redis is fast partly because it avoids threads: one loop, no locks, and the kernel (epoll) tells it exactly which sockets need attention. Good to know: newer Redis versions can use extra I/O threads for network reading/writing, but **commands still execute on the single main thread**.

5. The In-Memory Storage Engine

5.1 Redis is not a plain hashmap of strings

A common assumption is that Redis stores data like an `unordered_map<string, string>`. **That is incorrect**. Redis stores data in **optimized formats** chosen for **fast lookup and low memory use**. There is also **no single fixed internal representation per data type**: storing an integer does not mean it is always stored as an int.

EXAMPLE

In most programming languages an integer is always an int. In Redis the same logical value can be stored as an integer, an embedded string or a raw string depending on its size and content, and Redis switches between these **encodings automatically** as data grows. The client sees none of this. The result is roughly **5x to 10x less memory** than a naive hashmap implementation.

5.2 The redisObject wrapper

Every value (string, list, sorted set...) is wrapped in a **redisObject** with these fields:

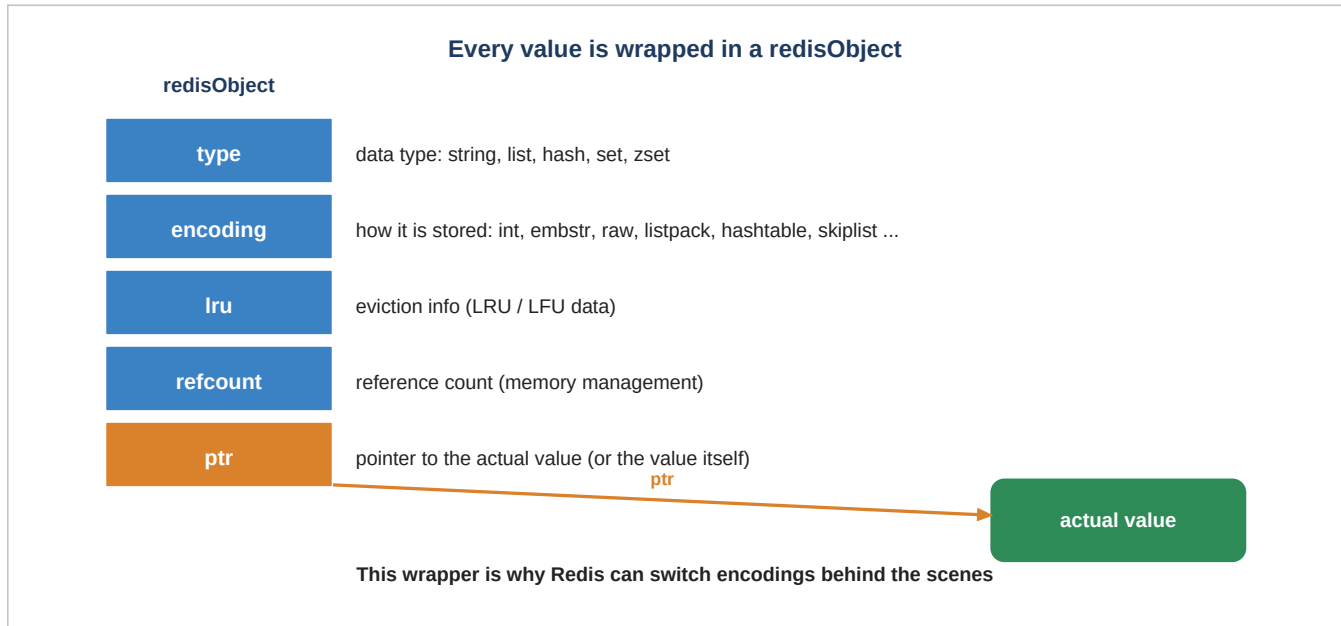


Figure 14: The *redisObject* structure that wraps every value

Field	Purpose	Example
type	The logical data type	string, list, hash, set, zset
encoding	How the value is physically stored	int, embstr, raw, listpack, hashtable, skiplist
lru	Bookkeeping for the eviction policy (LRU/LFU)	last access info
refcount	Reference count for memory management	how many references point at this object
ptr	Pointer to the actual value (or the value itself)	pointer to a string buffer, or the integer 42

5.3 String encodings by example

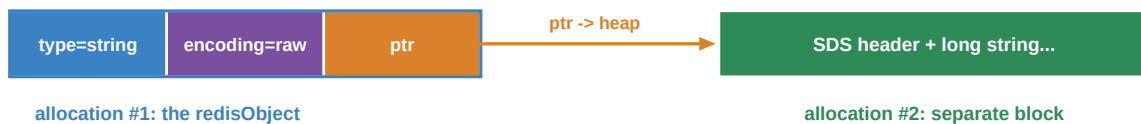
1) Integer 42 -> encoding = int



2) Short string 'hello' (<= 44 bytes) -> encoding = embstr



3) Long string (> 44 bytes) -> encoding = raw



Rule of thumb: <= 44 bytes: 1 allocation (embstr). > 44 bytes: 2 allocations (raw).

SDS = Simple Dynamic String, Redis's own efficient string structure

Redis picks the encoding itself and switches automatically; the client never sees it

Result: roughly 5x - 10x less memory than a naive hashmap-of-strings design

Figure 15: int, embstr and raw encodings for strings

(a) Integer: encoding = int

For the value **42**, Redis stores the number **directly inside the pointer field** of the redisObject. **No separate memory allocation** is needed; only the object itself is allocated.

(b) Short string (up to 44 bytes): encoding = embstr

For 'hello', the string is stored **right next to the redisObject in the same memory block** (embedded string). That is **one allocation**, which is also friendly to the CPU cache.

(c) Long string (over 44 bytes): encoding = raw

For a long string, Redis allocates the redisObject **and** a separate **SDS** block that holds the string. That is **two allocations**. **SDS (Simple Dynamic String)** is Redis's core structure for managing strings efficiently.

EXAMPLE

'42' -> int (0 extra allocations). 'hello' (5 bytes) -> embstr (1 allocation). A 300-character article snippet -> raw (2 allocations). The boundary is 44 bytes.

5.4 Hashes: listpack first, hash table later

Suppose you store user data in a hash. A small hash starts as a **listpack**: a compact packed sequence of field and value pairs in contiguous memory (very memory efficient, scanned linearly).

EXAMPLE

HSET user1 name Mayank age 29 and HSET user2 name Bob age 22. Each user is stored as a tiny packed list like [name, Mayank, age, 29]. With only a few fields, scanning it is instant and wastes almost no memory.

When the hash grows past a threshold, Redis automatically **converts it to a real hash table with $O(1)$ lookup**. Two **independent** triggers (either one is enough):

Trigger (default)	Meaning	Example
More than 128 entries	Hash has too many fields to scan quickly	user1 has 128 fields; adding field 129 (say Instagram id) triggers the upgrade
Any value over 64 bytes	One field is large, regardless of field count	user with only 2 fields (name, address) but a very long address > 64 bytes is upgraded immediately

EXAMPLE

A hash with just **5 fields** where one value is a **200-character string** is upgraded to a hash table even though it has only five fields.

5.5 Sorted sets

Sorted sets (leaderboards) work the same way: a **small** sorted set is a **listpack** (rank queries are $O(n)$, fine when tiny); a **large** one becomes a **skip list plus an internal hash table** (fast ordered access plus fast lookup by member).

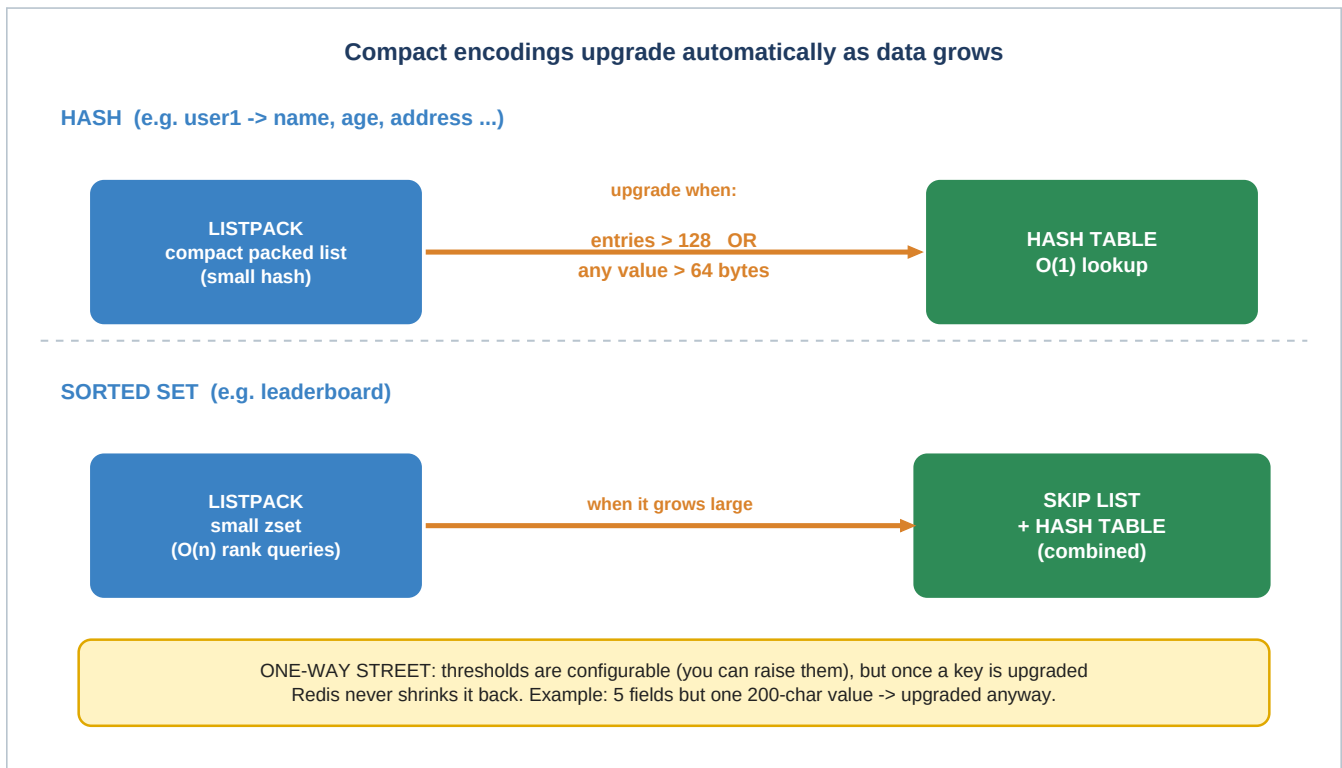


Figure 16: Hash and sorted-set encodings upgrade automatically as data grows

EXAMPLE

A game starts with 20 players: its leaderboard is a listpack. After the game goes viral with 2 million players, the leaderboard is automatically a skip list + hash table: you can fetch 'top 10' and 'rank of player X' quickly.

5.6 Thresholds are configurable, but upgrades are one-way

WATCH OUT

The thresholds (128 entries, 64 bytes) are **configurable**, but you can only **increase** them. Once a key has been upgraded to the full encoding, Redis **never downgrades** it; there is no shrink path. For example, if you raise the entry limit to 129 you cannot later go back to 128 and expect existing keys to shrink.

6. Persistence: RDB, AOF and Hybrid

Redis lives in RAM, and RAM is volatile. If the power goes out, the process crashes or the OS kills it, **everything is lost**. **Persistence** answers: **how much data can I recover after a failure?** You choose a technique according to how much loss you can tolerate. With **no persistence** (as in the source video's default assumption), any crash means total data loss.

6.1 RDB: point-in-time snapshots

RDB saves a **snapshot** of the whole data set at a point in time. How it works:

- When an RDB snapshot fires, Redis calls **fork()**, creating a **child process** that is an exact copy of the parent's memory view.
- The **child** walks through all keys, serializes them into a binary file called `dump.rdb`.
- The **parent keeps serving clients** as if nothing happened.

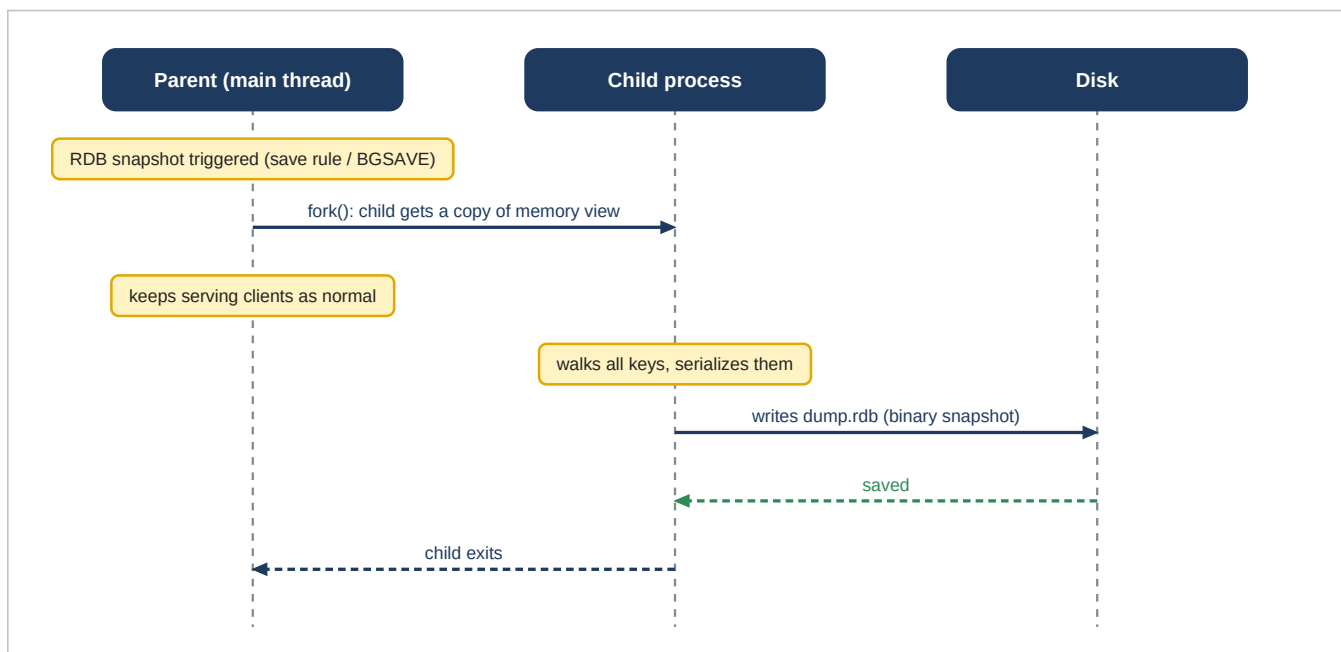


Figure 17: RDB snapshot: fork, child writes dump.rdb, parent keeps serving

Why the parent is not blocked: copy-on-write (COW)

After fork the parent and child **share the same memory pages**. When the parent modifies a page, the OS makes a **private copy of that page for the parent**, while the child still sees the original. So the child is **frozen in time at the moment of the fork** and gets a **consistent snapshot without any locking**.

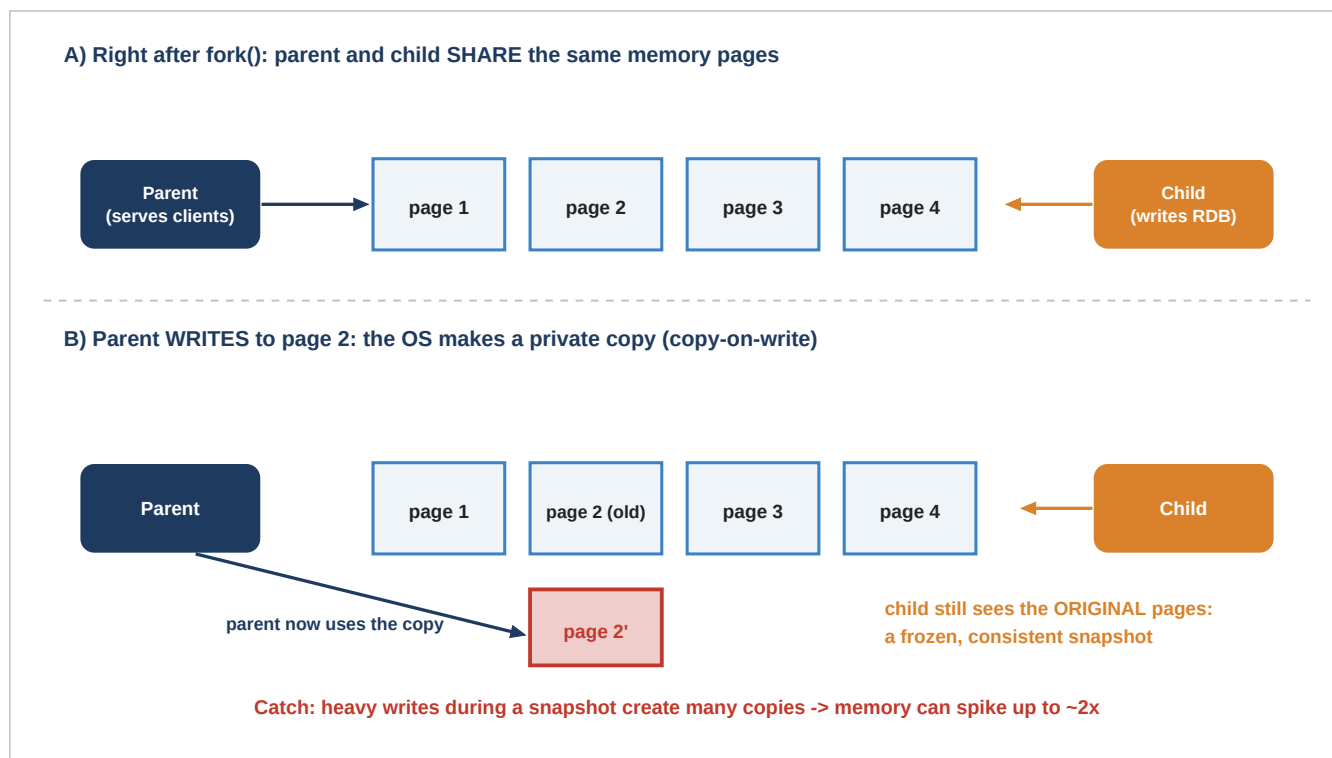


Figure 18: Copy-on-write keeps the child's snapshot consistent while the parent continues

Catches of RDB

- **Memory spike:** if the parent is **write-heavy** during the snapshot, many private page copies pile up and memory use can spike, sometimes to about **2x**. Example: a busy leaderboard being updated thousands of times per second while the snapshot runs can nearly double RAM use.
- **Data loss window:** you can only recover **up to the last snapshot**.

RDB: snapshots every 5 minutes



Recover only up to the last snapshot (min 10); everything after it is gone.

AOF (fsync every second): every write is logged



At most ~1 second of writes lost (ticks = fsync each second).

Figure 19: RDB loses everything since the last snapshot; AOF loses about a second

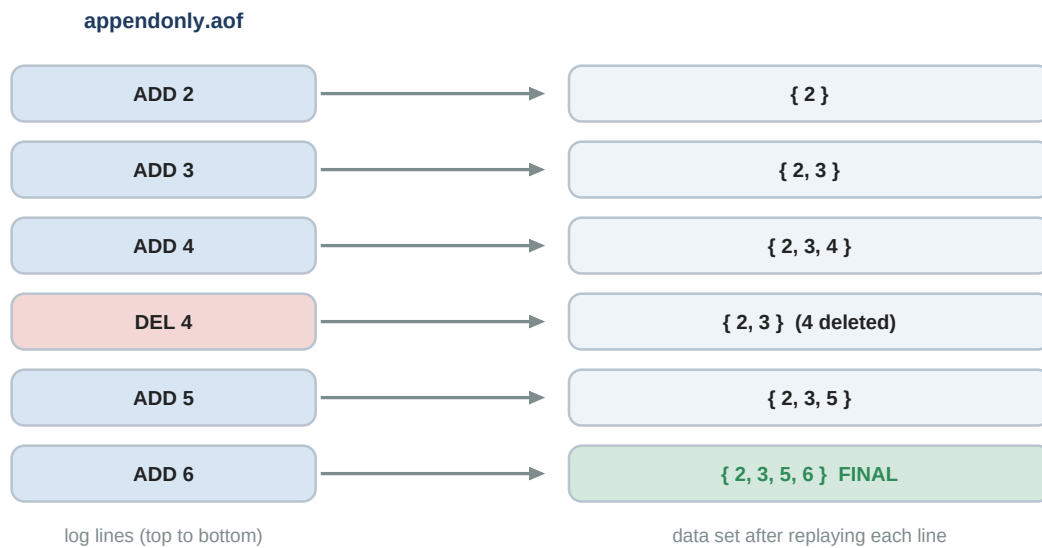
EXAMPLE

Snapshots every 5 minutes: at 0, 5, 10, 15 min. The server crashes at minute 14. You recover the minute-10 snapshot and **lose 4 minutes of data**.

6.2 AOF: append-only file

Instead of point-in-time snapshots, **AOF logs every write operation** Redis executes. Commands are **appended** to a file (hence append-only), stored in the same RESP format. On restart, Redis **replays the file from top to bottom** and rebuilds the whole data set.

AOF: append every write command to a log, replay it on restart



After a crash Redis replays the file from the top and rebuilds the full data set.

(The real file stores each command in RESP format, e.g. *3 \$3 SET ...)

Figure 20: AOF replay: insert 2, 3, 4, delete 4, insert 5, 6 gives { 2, 3, 5, 6 }

EXAMPLE

Log: add 2, add 3, add 4, delete 4, add 5, add 6. After a crash Redis replays: {2} -> {2,3} -> {2,3,4} -> delete 4 -> {2,3} -> {2,3,5} -> {2,3,5,6}. Both inserts and deletes are logged, so the final state is exact.

fsync policies: how often to flush the log to disk

AOF fsync policies: safety vs speed

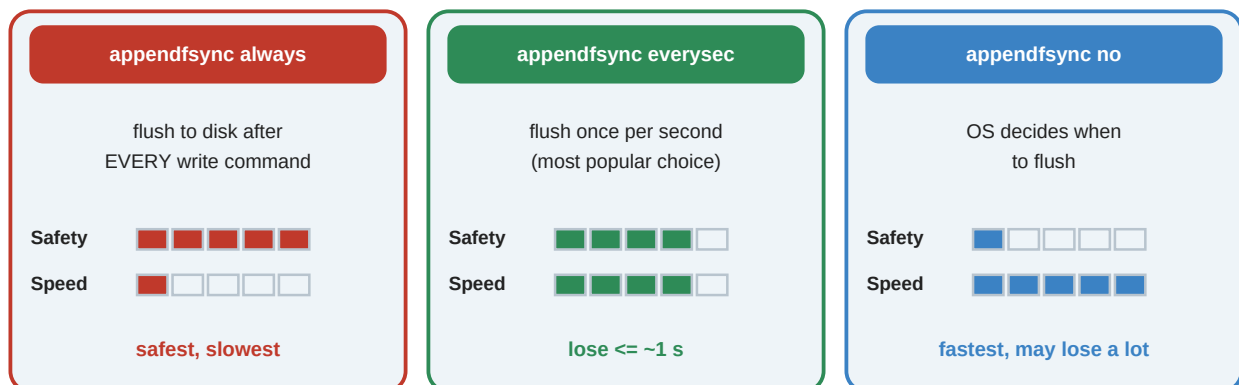


Figure 21: appendfsync: always vs everysec vs no

Policy	What happens	Trade-off
always	After every write command Redis calls fsync, forcing the OS to flush the file to disk	Safest, but slowest
everysec	Flush the accumulated writes once per second	Most popular: good balance, lose at most about 1 second
no	Let the OS decide when to flush	Fastest, but you can lose a lot of data

6.3 Hybrid RDB + AOF (Redis 7)

The hybrid mode **starts with AOF and every N seconds takes an RDB snapshot**. On recovery you **load the latest snapshot** and replay **only the commands after it**, instead of replaying every command since the very beginning.

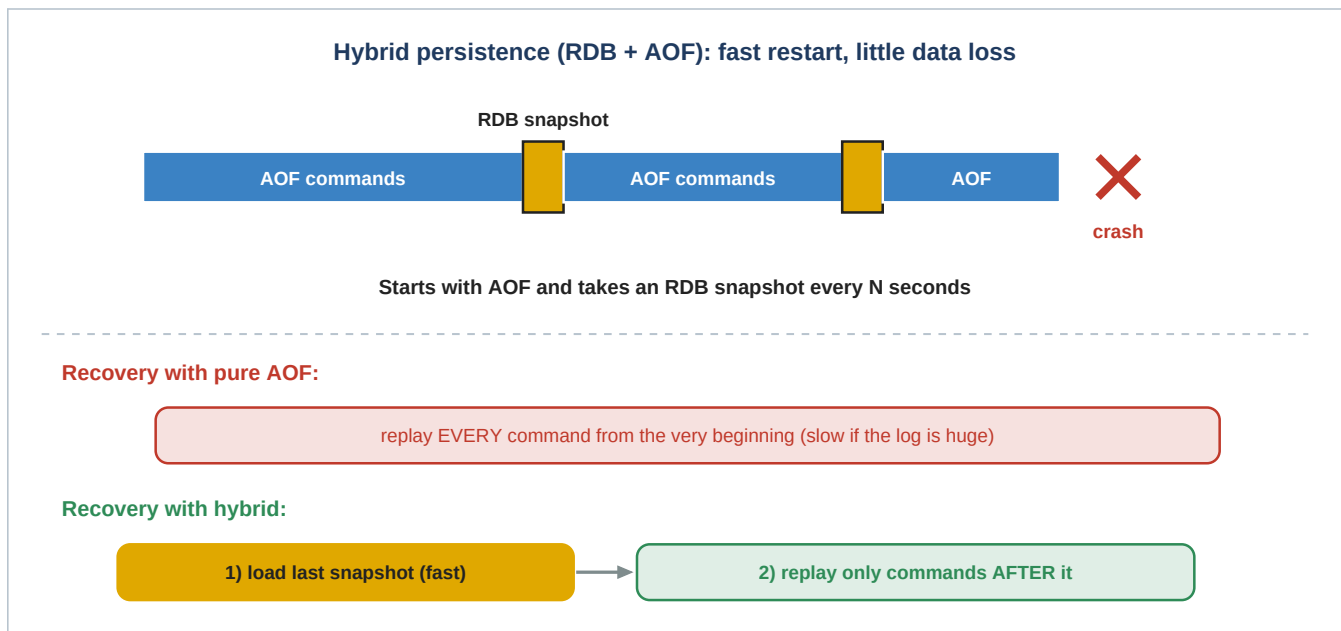


Figure 22: Hybrid recovery: load snapshot, then replay only the recent tail

EXAMPLE

After 3 days a pure AOF might contain 500 million commands, so replay is slow. With hybrid, Redis loads the snapshot taken a few seconds before the crash and replays only the handful of commands written since: fast restart and tiny loss.

6.4 Comparison and how to choose

Aspect	No persistence	RDB	AOF	Hybrid
Data loss on crash	Everything	Since last snapshot (e.g. minutes)	~1 s (everysec) or none (always)	Very little
Restart speed	Instant (empty)	Fast (load one file)	Slower (replay log)	Fast
Write overhead	None	Fork + possible memory spike	Logging cost	Both, moderate
Good for	Pure disposable cache	Backups, tolerable loss	Strong durability	Best of both

EXAMPLES

Pure cache of product pages: no persistence is fine; it re-warms from the DB. **Session store:** RDB or AOF everysec so users are not all logged out after a restart. **Leaderboard you cannot lose:** AOF everysec or hybrid.

7. End-to-End Architecture Walkthrough

Now we combine every component. Follow what happens when an application runs SET user1 Alice.

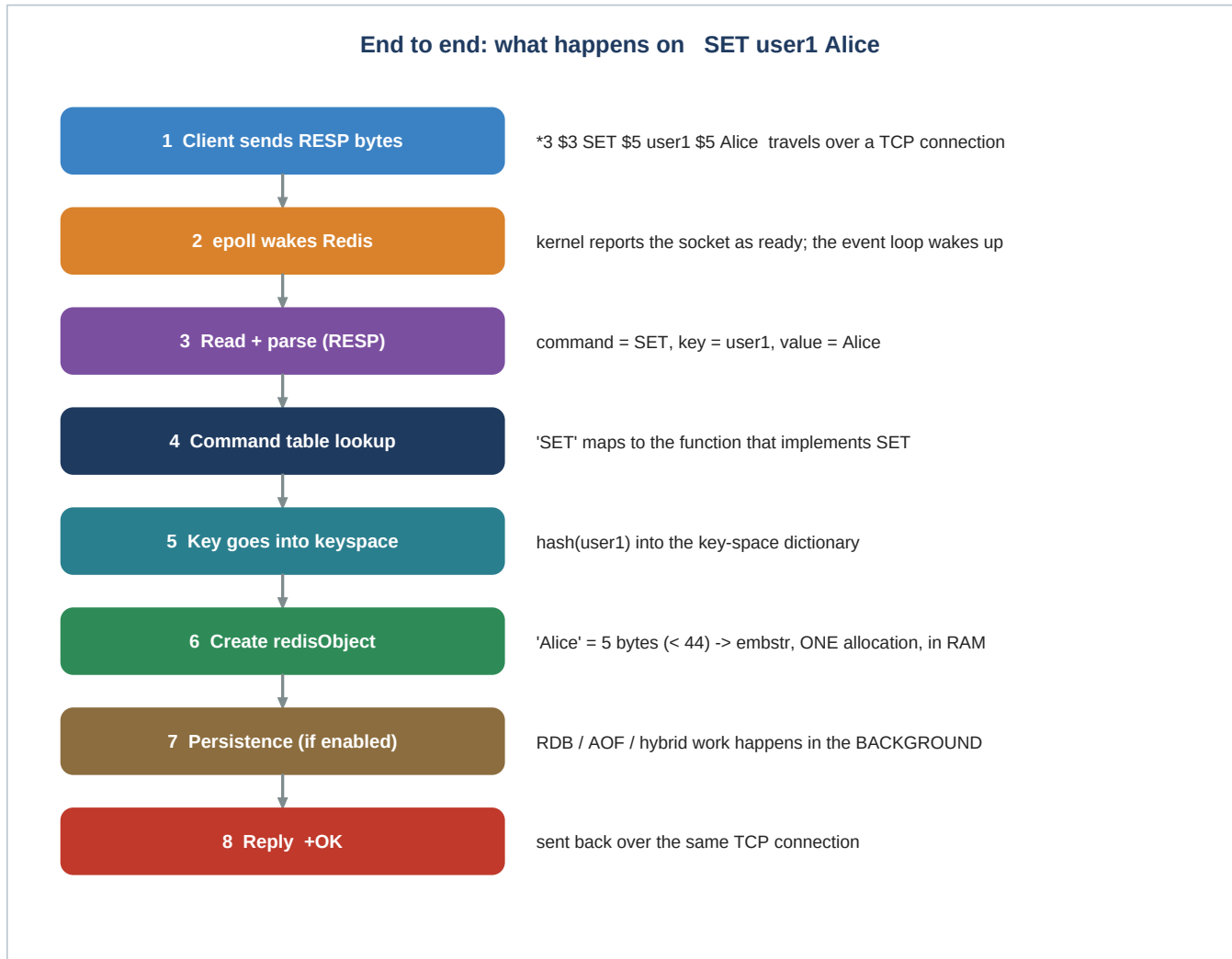


Figure 23: The full life of one SET command

- **1. Client call:** the app sends the command; the client library encodes it in **RESP** and sends it over TCP.
- **2. epoll wakes Redis:** the kernel reports the socket as ready; the **event loop** wakes up.
- **3. Read and parse:** the RESP layer parses the bytes: command SET, key user1, value Alice.
- **4. Command table lookup:** Redis has a table mapping each command name to the function implementing it; 'SET' maps to the SET implementation.
- **5. Keyspace:** the key user1 is hashed into the **key-space dictionary**.
- **6. Create the value object:** a redisObject is created for 'Alice'. It is 5 bytes (under 44), so it uses **embstr** with a **single allocation**, and lives in RAM.

- **7. Persistence (if enabled):** RDB, AOF or hybrid work happens **in the background**, not blocking the reply.
- **8. Response:** Redis sends +OK\r\n back over the same TCP connection.

EXAMPLE

Why it is fast: no thread switches (single event loop), no wasted polling (epoll), a tiny text protocol (RESP), compact in-memory encodings (embstr, listpack), and slow disk work pushed to the background.

8. Lazy Free

Consider a **delete** on a key that holds something huge, for example a sorted set with **100,000 members** (or millions of records). Freeing that memory **synchronously** would **block the event loop for milliseconds**, delaying every other client's response, because the main thread is single-threaded.

Lazy free solves this: Redis **unlinks the key from the key space** immediately and returns 'OK' to the caller, and a **background thread** actually frees the memory later.

A) DEL on a huge key (100,000-member sorted set): freed synchronously



event loop **BLOCKED** for milliseconds: every other client waits

B) UNLINK / lazy free: cheap unlink, memory freed in a background thread



Key is removed from the key space at once and the reply 'OK' is immediate;
command execution still stays single-threaded

Figure 24: Synchronous delete blocks the event loop; lazy free hands the work to a background thread

EXAMPLE

A cache holds a huge 'trending posts' sorted set. An admin deletes it. Without lazy free, for a few milliseconds no other client gets a reply (login checks time out). With lazy free the delete returns instantly and other requests continue; the memory is reclaimed in the background.

In Redis this is available as the UNLINK command (a non-blocking DEL) and lazy-free configuration options.

KEY TAKEAWAY

Redis still has **one thread for command processing**. Background threads exist only for slow cleanup work like this, so there is no thread-synchronization overhead on the main path.

9. Big Picture, Cheat Sheet and Interview Q&A

9.1 Architecture at a glance

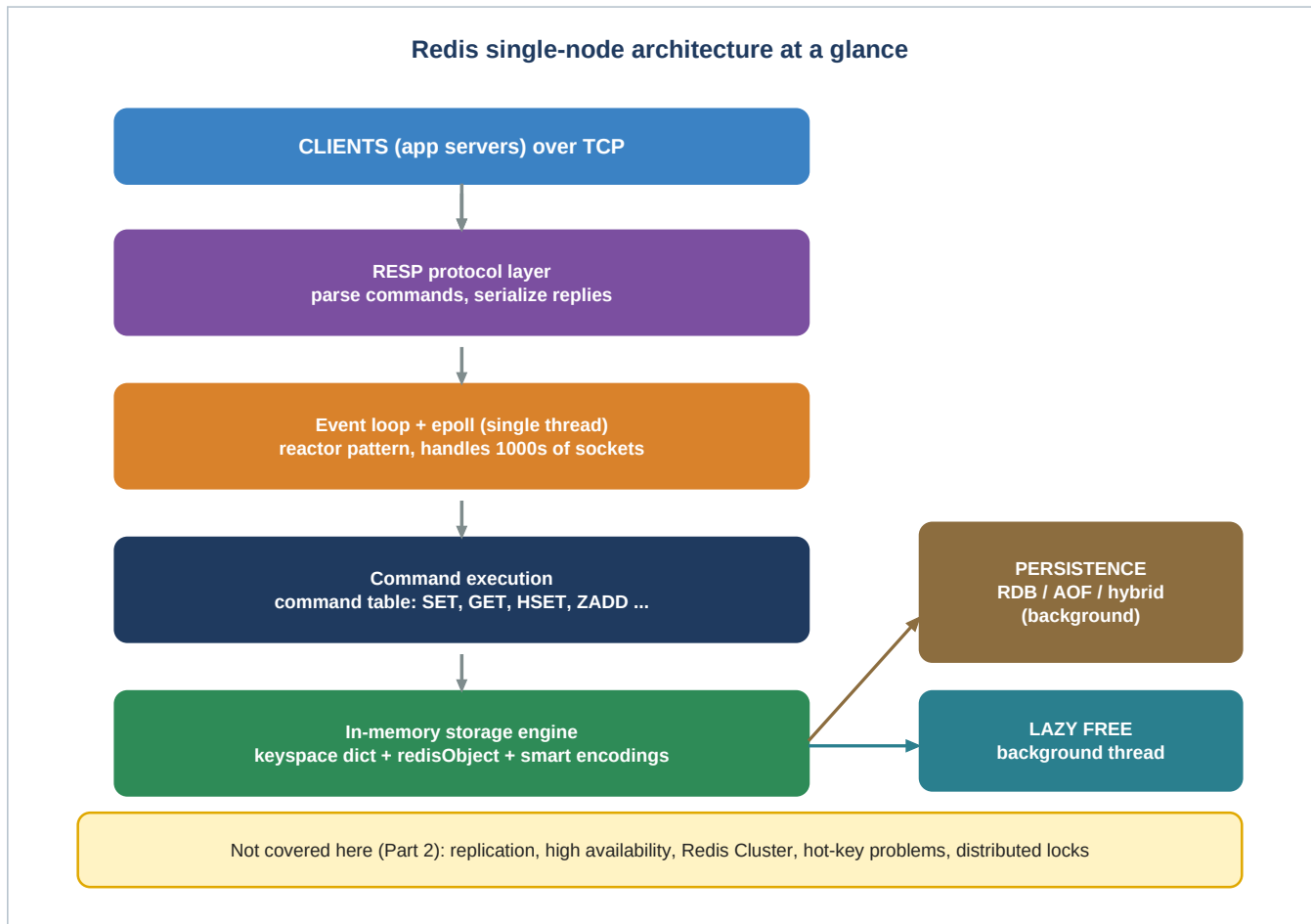


Figure 25: Single-node Redis: protocol, event loop, execution, storage, persistence, lazy free

Everything above describes a **single node**. Redis also works in distributed environments; replication, high availability, Redis Cluster, hot-key problems and distributed locks are the topics of a follow-up (Part 2).

9.2 One-line summary per component

Component	Remember this
In-memory store	All data in RAM = very low latency. Cache layer, not primary DB (volatile).
Capacity	100K-500K ops/s per instance, < 1 ms latency, data size limited by RAM (64 GB -> ~60 GB).
Eviction	LRU for sessions, LFU for frequency; no eviction + over-provision for leaderboards/counters.
RESP	Text wire protocol over TCP: prefixes + - : \$ *, lines end with \r\n.
Event loop + epoll	Single thread; kernel reports ready sockets; loop = epoll_wait, read, parse, execute, reply.
Why not threads	100,000 threads x 1 MB = 100 GB RAM, plus context switching and locks.
Storage engine	redisObject wrapper; encodings int / embstr (<= 44 B) / raw (> 44 B).
Encoding upgrades	listpack -> hash table at > 128 entries or value > 64 B; zset -> skiplist + hash table; no downgrade.
RDB	Fork + copy-on-write snapshot (dump.rdb); loses data since last snapshot; memory spike under heavy writes.
AOF	Log every write, replay on restart; fsync always / everysec (popular) / no.
Hybrid	AOF + periodic RDB: load snapshot, replay only the tail.
Lazy free	Unlink instantly, free memory in a background thread; main thread never blocked.

9.3 Interview Q&A

Q: What is Redis?

A: An in-memory key-value store that keeps all data in RAM for extremely low latency and high speed.

Q: Why do we need Redis?

A: To get low latency and fast speed; it sits as a cache between the app and the database.

Q: Can Redis be your primary database?

A: Not by design: data is in volatile RAM. Use it as a cache layer (persistence reduces but does not remove the risk).

Q: What throughput and latency does one instance give?

A: Roughly 100K-500K ops/sec and under 1 ms per query.

Q: How much data can an instance hold?

A: As much as fits in RAM minus OS needs (about 60 GB on a 64 GB node).

Q: What if the data exceeds RAM?

A: Use an eviction policy (LRU/LFU), or over-provision when keys must never be dropped.

Q: What is RESP?

A: Redis Serialization Protocol: simple text wire format with type prefixes (+ - : \$ *) and \r\n terminators.

Q: Why is Redis single-threaded?

A: Thread-per-client would cost huge memory (100,000 x 1 MB = 100 GB) and add locking and context-switch overhead.

Q: How does one thread handle thousands of connections?

A: epoll: the kernel watches sockets and wakes Redis only when some are ready; the event loop then serves them.

Q: epoll vs event loop?

A: epoll is the Linux readiness API; the event loop is the reactor pattern that calls it, then reads, parses, executes and replies.

Q: How does Redis save memory?

A: Compact encodings chosen automatically (int in pointer, embstr, listpack, skiplist) - 5x-10x less than a naive hashmap.

Q: When does a hash become a hash table?

A: More than 128 entries or any value over 64 bytes (defaults); it never downgrades.

Q: RDB vs AOF?

A: RDB: periodic snapshots via fork + COW, can lose minutes. AOF: logs every write, replay on restart, lose about 1 s with everysec.

Q: What is copy-on-write?

A: After fork, pages are shared; a page the parent changes is copied privately, so the child keeps a frozen consistent snapshot.

Q: What is hybrid persistence?

A: AOF plus periodic RDB: restore the snapshot and replay only commands after it.

Q: What is lazy free?

A: Unlink a huge key immediately and free its memory in a background thread so the event loop is not blocked.

9.4 Notes on the source material

- The transcript is auto-generated, so some terms are garbled: 'radius' = Redis, 'RSP' = RESP, 'eole / e pole weight' = epoll / epoll_wait, 'embr' = embstr, 'list spec' = listpack, 'melo / maloc' = malloc. These notes use the correct terms.
- In the RESP example the speaker says the key user1 has six characters, but it has **five** (u, s, e, r, 1), so the correct encoding is \$5\r\nuser1\r\n, as shown here.
- The video states that persistence is off by default. In practice a stock Redis configuration commonly ships with periodic RDB snapshot rules enabled, so check your own redis.conf; the notes follow the video for the teaching point (without any persistence, a crash loses everything).
- 'Unsigned LRU' in the redisObject refers to the small bookkeeping field that stores LRU/LFU information.
- The AOF replay example uses simple add/delete operations on values to illustrate replay; real AOF files store RESP-encoded commands.
- This is the single-node architecture. Replication, high availability, cluster, hot keys and distributed locks are mentioned as a future Part 2.